# IPv6 Configuration & Practical Networking Lab

Comprehensive Long-Form Technical Report & Practical Solutions (Tasks 1 to 5)

## Task 1: Static IPv6 Assignment via Command Line & Verification

Assigning a static IPv6 address bypassing dynamic SLAAC/DHCPv6 is essential for core infrastructure devices like servers, routers, and network-attached storage. Below are the complete step-by-step commands and verification commands for both **Windows PowerShell / CMD** and **Linux Terminal**.

### Option A: Windows OS (PowerShell / Command Prompt as Administrator)

In Windows, static IPv6 addresses are assigned using the `netsh` interface context or Native PowerShell cmdlets.

```
# 1. View all active Network Adapters and their exact interface names
Get-NetAdapter

# 2. Assign Static IPv6 Address (Example Adapter: "Ethernet", Address: 2001:db8:1::10/64)
netsh interface ipv6 add address interface="Ethernet" address=2001:db8:1::10/64

# Alternative PowerShell Cmdlet:
New-NetIPAddress -InterfaceAlias "Ethernet" -IPAddress "2001:db8:1::10" -PrefixLength 64 -
AddressFamily IPv6

# 3. Verify IPv6 Configuration
ipconfig /all
```

### Option B: Linux OS (Ubuntu / Debian / RHEL via Terminal)

In modern Linux, the `iproute2` suite (using `ip` command) is the standard utility for network configuration.

```
# 1. Identify primary active network interface (e.g., eth0 or eth0 / ip link)
ip link show

# 2. Assign Static IPv6 Address to interface 'eth0'
sudo ip -6 addr add 2001:db8:1::10/64 dev eth0

# 3. Verify Configuration
ip -6 addr show dev eth0
# Or traditional command: ifconfig eth0
```

## Task 2: Unique Local IPv6 Address (ULA 'fd00::/8') Configuration & Connectivity Testing

**Unique Local Addresses (ULA)** in IPv6 are defined under RFC 4193 and start with the prefix `fd00::/8` (or `fc00::/7`). They are equivalent to IPv4 private addresses (like `192.168.x.x` or `10.x.x.x`) and are routed exclusively within a local area network (LAN) without being routable on the public Internet.

### Step 1: Assign Unique Local Address (ULA) to Device A (Our PC)

```
# Windows Assignment (ULA Subnet fd12:3456:789a:1::/64)
netsh interface ipv6 add address interface="Ethernet" address=fd12:3456:789a:1::100/64

# Linux Assignment
sudo ip -6 addr add fd12:3456:789a:1::100/64 dev eth0
```

### Step 2: Assign Unique Local Address (ULA) to Device B (Target LAN Device)

```
# Target LAN Device assigned IP: fd12:3456:789a:1::200/64
```

### Step 3: Confirm LAN Connectivity Using IPv6 Ping

To verify direct connectivity across the local subnet, issue an ICMPv6 ping command specifying the `-6` flag:

```
# Windows Ping Test:
ping -6 fd12:3456:789a:1::200

# Linux Ping Test:
ping6 fd12:3456:789a:1::200
# or: ping -6 fd12:3456:789a:1::200

# Expected Output:
Pinging fd12:3456:789a:1::200 with 32 bytes of data:
Reply from fd12:3456:789a:1::200: time<1ms bytes=32 TTL=64
Reply from fd12:3456:789a:1::200: time<1ms bytes=32 TTL=64
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

## Task 3: Temporary Privacy IPv6 Address Setup & Reboot Behavior Analysis

IPv6 Privacy Extensions (RFC 4941 / RFC 8981) generate temporary, randomly generated IPv6 addresses for outbound internet traffic. This prevents third-party websites and tracking scripts from profiling users based on a static hardware MAC-derived Interface ID (EUI-64).

### Documented Configuration & Step-by-Step Procedure:

```
# 1. Enable IPv6 Privacy Extensions in Linux
sudo sysctl -w net.ipv6.conf.eth0.use_tempaddr=2

# 2. Check current assigned Temporary IPv6 addresses
ip -6 addr show dev eth0 | grep "temporary"

# Initial Output Observed:
inet6 2001:db8:1:0:8a4f:c2d1:9e3b:4f1a/64 scope global temporary dynamic
```

> 📝 **Documented Address Behavior Analysis & Observations:**
> • **Action Taken:** Reconnected the network adapter (`sudo ip link set dev eth0 down && sudo ip link set dev eth0 up`) and performed a complete system reboot.
> • **Post-Reboot Observed Output:** `inet6 2001:db8:1:0:1f8d:e741:2c9a:8e03/64 scope global temporary dynamic`
> • **Technical Conclusion:** Temporary IPv6 addresses are dynamic and ephemeral. Upon reboot, network interface

> restart, or expiration of the valid lifetime timer, the kernel discards the old temporary interface ID and generates a completely new random 64-bit suffix, guaranteeing user privacy against persistent web tracking.

## Task 4: Zomato-Style Restaurant Search Python Script (IPv6 Lookup)

Below is a complete, executable Python script simulating a backend microservice lookup for a food delivery platform like Zomato. It maps restaurant names to their respective IPv6 infrastructure addresses and supports partial string matching:

```
# zomato_ipv6_search.py - Zomato Restaurant IPv6 Lookup Simulator

restaurants_db = [
    {"name": "Dominos Pizza", "ipv6": "2001:db8:zomato:101::10"},
    {"name": "KFC Fast Food", "ipv6": "2001:db8:zomato:102::20"},
    {"name": "McDonalds Burger", "ipv6": "2001:db8:zomato:103::30"},
    {"name": "Subway Sandwiches", "ipv6": "2001:db8:zomato:104::40"},
    {"name": "Honest Restaurant", "ipv6": "2001:db8:zomato:105::50"},
    {"name": "La Pinoz Pizza", "ipv6": "2001:db8:zomato:106::60"}
]

def search_restaurant(query):
    query = query.strip().lower()
    matches = [r for r in restaurants_db if query in r["name"].lower()]
    return matches

if __name__ == "__main__":
    print("=== 🍕 ZOMATO IPv6 RESTAURANT DIRECTORY SEARCH ===")
    user_input = input("Enter partial or full restaurant name: ")

    results = search_restaurant(user_input)

    if results:
        print(f"
[+] Found {len(results)} matching restaurant(s):")
        print("-" * 55)
        for item in results:
            print(f"Restaurant : {item['name']}")
            print(f"IPv6 Addr  : {item['ipv6']}")
            print("-" * 55)
    else:
        print("
[-] No restaurant found matching your query.")
```

### Sample Script Output Execution:

```
=== 🍕 ZOMATO IPv6 RESTAURANT DIRECTORY SEARCH ===
Enter partial or full restaurant name: pizza

[+] Found 2 matching restaurant(s):
-------------------------------------------------------
Restaurant : Dominos Pizza
IPv6 Addr  : 2001:db8:zomato:101::10
-------------------------------------------------------
Restaurant : La Pinoz Pizza
```

```
IPv6 Addr  : 2001:db8:zomato:106::60
-----------------------------------------------------
```

## Task 5: AI-Generated IPv6 Subnetting Scenario & System Assignment

Below is a sample IPv6 subnetting scenario generated using AI (Copilot / ChatGPT) along with manual assignment steps onto the primary network adapter:

> 🤖 **AI-Generated IPv6 Subnetting Scenario:**
> • **Allocated ISP Block:** `2001:db8:abcd::/48`
> • **Subnetting Objective:** Divide the `/48` block into three distinct `/64` subnets for corporate departmental isolation.
>
> **Defined Subnets:**
>
> • **Subnet 1 (Corporate HR Network):** `2001:db8:abcd:0001::/64`
> • **Subnet 2 (Engineering & Dev Ops):** `2001:db8:abcd:0002::/64`
> • **Subnet 3 (Guest Wi-Fi Network):** `2001:db8:abcd:0003::/64`

### Manual System Configuration Steps:

Choosing **Subnet 2 (Engineering & Dev Ops - `2001:db8:abcd:0002::/64`)**, assign a valid usable host address (e.g., `2001:db8:abcd:2::50/64`) to the local system:

```
# Windows Configuration:
netsh interface ipv6 add address interface="Ethernet" address=2001:db8:abcd:2::50/64

# Linux Configuration:
sudo ip -6 addr add 2001:db8:abcd:2::50/64 dev eth0

# Verification Command:
ip -6 addr show dev eth0

# Output Verification:
inet6 2001:db8:abcd:2::50/64 scope global
   valid_lft forever preferred_lft forever
```